



Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"



ESCUELA SUPERIOR DE CÓMPUTO

PRÁCTICA 01

Elliptic Curve

Residuos, puntos y aritmética de curvas

UNIDAD DE APRENDIZAJE
Selected Topics in Cryptography

GRUPO
7CM1

DOCENTE
Sandra Díaz Santiago

INTEGRANTES

Eduardo Alonso Sánchez

2023630155

Sesión: 8 de septiembre de 2026

Residuos y raíces

Se guarda $y^2 \bmod p$ en un arreglo; la otra raíz es $p - y$.

src/quadratic_residues.c

Líneas 1–54

```

1 #include "elliptic_curve.h"
2 #include <inttypes.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 /* Se supone que p es primo; no hacemos una prueba de primalidad. */
7 int quadratic_residues(uint32_t p, uint32_t **roots, FILE *output) {
8     *roots = NULL;
9     if (p <= 3 || p % 2 == 0) {
10         fprintf(stderr, "Error: p must be a prime greater than 3.\n");
11         return 1;
12     }
13     /* Reservamos una entrada por residuo; quien llama libera la tabla. */
14     if ((uint64_t)p * sizeof(**roots) > SIZE_MAX ||
15         !(*roots = malloc((size_t)p * sizeof(**roots)))) {
16         fprintf(stderr, "Error: insufficient memory for the residue table.\n");
17         return 1;
18     }
19     /* UINT32_MAX marca los residuos que no tienen raíz. */
20     memset(*roots, 0xff, (size_t)p * sizeof(**roots));
21     /* Solo recorreremos hasta p/2: la otra raíz de y es p-y. */
22     for (uint32_t y = 0; y <= p / 2; ++y) {
23         uint32_t r = (uint32_t)((uint64_t)y * y % p);
24         (*roots)[r] = y;
25     }
26     /* La función de la curva usa output = NULL para no imprimir aquí. */
27     if (output) {
28         fprintf(output, "p = %" PRIu32 "\nQR_p (nonzero squares) = {", p);
29         const char *separator = "";
30         for (uint32_t r = 1; r < p; ++r) {
31             if ((*roots)[r] == UINT32_MAX) continue;
32             fprintf(output, "%s%" PRIu32, separator, r);
33             separator = ", ";
34         }
35         fprintf(output, "}\nResidue -> square roots modulo p\n");
36         /* El cero tiene una sola raíz; los demás cuadrados tienen dos. */
37         fprintf(output, "0 -> {0} (zero is listed separately)\n");
38         for (uint32_t r = 1; r < p; ++r) {
39             uint32_t y = (*roots)[r];
40             if (y != UINT32_MAX)
41                 fprintf(output, "%" PRIu32 " -> {" PRIu32 " ", " PRIu32 " }\n",
42                     r, y, p - y);
43         }
44     }
45     return 0;
46 }
47
48 /* Permite probar el primer ejercicio usando solamente p. */
49 int print_quadratic_residues(uint32_t p) {
50     uint32_t *roots;
51     int status = quadratic_residues(p, &roots, stdout);
52     free(roots);
53     return status;
54 }

```

Evaluación y salida de puntos

Para cada x , se busca la raíz de $x^3 + ax + b \bmod p$ en el arreglo.

src/elliptic_curve.c

Líneas 1–28

```

1 #include "elliptic_curve.h"
2 #include <inttypes.h>
3 #include <stdlib.h>
4
5 /* Calculamos  $x^3 + ax + b$  modulo  $p$  usando productos de 64 bits. */
6 static uint32_t rhs(uint32_t x, uint32_t p, uint32_t a, uint32_t b) {
7     uint64_t x2 = (uint64_t)x * x % p;
8     return (uint32_t)((x2 * x % p + (uint64_t)a * x % p + b) % p);
9 }
10
11 /* La misma salida sirve para la pantalla y para el archivo. */
12 static void write_points(FILE *out, uint32_t p, uint32_t a, uint32_t b,
13                          const uint32_t *roots, uint64_t count) {
14     fprintf(out, "p = %" PRIu32 " \na = %" PRIu32 " \nb = %" PRIu32 " \n", p, a, b);
15     fprintf(out, "E:  $y^2 = x^3 + ax + b \pmod{p}$  \n");
16     fprintf(out, "Rational points (x, y, z): \n");
17     for (uint32_t x = 0; x < p; ++x) {
18         uint32_t y = roots[rhs(x, p, a, b)];
19         /* Si el resultado no tiene raíz, este x no da un punto. */
20         if (y == UINT32_MAX) continue;
21         fprintf(out, "(%" PRIu32 " , %" PRIu32 " , 1) \n", x, y);
22         /* Con  $y = 0$  evitamos escribir el mismo punto dos veces. */
23         if (y != 0)
24             fprintf(out, "(%" PRIu32 " , %" PRIu32 " , 1) \n", x, p - y);
25     }
26     fprintf(out, "(0, 1, 0) \nTotal points (including infinity): %" PRIu64 " \n",
27            count);
28 }

```

Uso de IA. Se usó ChatGPT/Codex para generar el código de src/ e include/, preparar pruebas y organizar el reporte. El apoyo se utilizó para construir y comprobar la aritmética con arreglos.

Conteo y archivo

Se cuentan los puntos y se agrega $(0, 1, 0)$ para el infinito.

src/elliptic_curve.c

Líneas 29–74

```

29 int rational_points(uint32_t p, uint32_t a, uint32_t b, const char *path) {
30     if (p <= 3 || p % 2 == 0 || a >= p || b >= p) {
31         fprintf(stderr, "Error: require prime p > 3 and 0 <= a,b < p.\n");
32         return 1;
33     }
34     /* Una curva no singular cumple  $4a^3 + 27b^2 \neq 0$  modulo p. */
35     uint64_t a3 = ((uint64_t)a * a % p) * a % p;
36     uint64_t b2 = (uint64_t)b * b % p;
37     if ((4 * a3 + 27 * b2) % p == 0) {
38         fprintf(stderr, "Error: singular curve;  $4a^3 + 27b^2 = 0 \pmod{p}$ .\n");
39         return 1;
40     }
41     /* Primero obtenemos los residuos y las raices del ejercicio 1. */
42     uint32_t *roots;
43     if (quadratic_residues(p, &roots, NULL)) return 1;
44     uint64_t count = 1; /* Contamos el punto al infinito desde el inicio. */
45     for (uint32_t x = 0; x < p; ++x) {
46         uint32_t y = roots[rhs(x, p, a, b)];
47         if (y != UINT32_MAX) count += y == 0 ? 1 : 2;
48     }
49     /* Guardamos p, a, b y todos los puntos en el archivo indicado. */
50     FILE *file = fopen(path, "w");
51     if (!file) {
52         fprintf(stderr, "Error: cannot open output file: %s\n", path);
53         free(roots);
54         return 1;
55     }
56     write_points(file, p, a, b, roots, count);
57     /* Revisamos que la escritura y el cierre hayan terminado bien. */
58     int failed = ferror(file);
59     if (fclose(file) != 0) failed = 1;
60     if (failed) {
61         fprintf(stderr, "Error: could not finish writing: %s\n", path);
62         free(roots);
63         return 1;
64     }
65     write_points(stdout, p, a, b, roots, count);
66     printf("Saved to: %s\n", path);
67     free(roots);
68     return 0;
69 }
70
71 /* Version con nombre de archivo fijo para usar desde el navegador. */
72 int print_rational_points(uint32_t p, uint32_t a, uint32_t b) {
73     return rational_points(p, a, b, "elliptic-curve-points.txt");
74 }

```

Generación de curvas

Se genera un candidato de n bits y se busca $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

src/curve_arithmetic.c

Líneas 1–41

```

1 #include "curve_arithmetic.h"
2
3 int curve_init(Curve *curve, Big p, Big a, Big b) {
4     if (!field_init(&curve->field, p) || big_compare(a,p)>=0 ||
5         big_compare(b,p)>=0)
6         return 0;
7     curve->a = a; curve->b = b;
8     const Field *f = &curve->field;
9     Big a3 = field_mul(f, field_mul(f,a,a), a), b2 = field_mul(f,b,b);
10    /* Sumamos 4a^3 y 27b^2; así las constantes también sirven si p es 5. */
11    Big discriminant = big_small(0);
12    for (unsigned i=0; i<4; ++i) discriminant = field_add(f,discriminant,a3);
13    for (unsigned i=0; i<27; ++i) discriminant = field_add(f,discriminant,b2);
14    return big_bits(discriminant) != 0;
15 }
16
17 int point_belongs(const Curve *curve, Point point) {
18     if (point.z == 0)
19         return !big_bits(point.x) && big_compare(point.y,big_small(1))==0;
20     const Field *f = &curve->field;
21     if (point.z != 1 || big_compare(point.x,f->p)>=0 || big_compare(point.y,f->p)>=0)
22         return 0;
23     Big rhs = field_mul(f,field_mul(f,point.x,point.x),point.x);
24     rhs = field_add(f,field_add(f,rhs,field_mul(f,curve->a,point.x)),curve->b);
25     return big_compare(field_mul(f,point.y,point.y),rhs)==0;
26 }
27
28 /* Fijamos el primer bit y el último: exactamente n bits y candidato impar. */
29 int generate_curve(unsigned bits, Curve *curve) {
30     if (bits < 3 || bits > BIG_BITS) return 0;
31     Big p, a, b;
32     do {
33         p = big_random(bits);
34         p.word[(bits-1)/32] |= UINT32_C(1) << ((bits-1)%32);
35         p.word[0] |= 1;
36     } while (!probable_prime(p));
37     do {
38         a = random_below(p); b = random_below(p);
39     } while (!curve_init(curve,p,a,b));
40     return 1;
41 }

```

C17 estándar. Los enteros usan arreglos de 32 bits. Miller–Rabin hace 32 rondas: el resultado es un primo probable. rand se usa para la práctica, no para generar claves de uso real.

Suma y duplicación

Se revisan los puntos, se calcula la pendiente y se obtiene el resultado módulo p .

src/curve_arithmetic.c

Líneas 42–86

```

42 static Point infinity(void) {
43     Point result = {big_small(0),big_small(1),0};
44     return result;
45 }
46
47 /* Con la pendiente calculamos  $x_3 = \lambda^2 - x_1 - x_2$ ,  $y_3 = \lambda(x_1 - x_3) - y_1$ . */
48 static Point from_slope(const Curve *curve, Point p, Point q, Big slope) {
49     const Field *f = &curve->field;
50     Point result;
51     result.x = field_sub(f,field_sub(f,field_mul(f,slope,slope),p.x),q.x);
52     result.y = field_sub(f,field_mul(f,slope,field_sub(f,p.x,result.x)),p.y);
53     result.z = 1;
54     return result;
55 }
56
57 int point_double(const Curve *curve, Point p, Point *result) {
58     if (!point_belongs(curve,p)) return 0;
59     if (!p.z || !big_bits(p.y)) { *result = infinity(); return 1; }
60     const Field *f = &curve->field;
61     Big x2 = field_mul(f,p.x,p.x);
62     Big numerator = field_add(f,field_add(f,x2,x2),x2);
63     numerator = field_add(f,numerator,curve->a);
64     Big denominator = field_add(f,p.y,p.y);
65     Big slope = field_mul(f,numerator,field_inverse(f,denominator));
66     *result = from_slope(curve,p,p,slope);
67     return 1;
68 }
69
70 int point_add(const Curve *curve, Point p, Point q, Point *result) {
71     if (!point_belongs(curve,p) || !point_belongs(curve,q)) return 0;
72     if (!p.z) { *result = q; return 1; }
73     if (!q.z) { *result = p; return 1; }
74     const Field *f = &curve->field;
75     /* Tambien atendemos  $P=Q$  y  $P=-Q$ , aunque el ejercicio los excluye. */
76     if (big_compare(p.x,q.x)==0) {
77         if (big_compare(p.y,q.y)==0) return point_double(curve,p,result);
78         *result = infinity(); return 1;
79     }
80     Big numerator = field_sub(f,q.y,p.y);
81     Big denominator = field_sub(f,q.x,p.x);
82     Big slope = field_mul(f,numerator,field_inverse(f,denominator));
83     *result = from_slope(curve,p,q,slope);
84     return 1;
85 }
86

```

Captura de consola

Captura de consola

```
$ ./build/elliptic-curve qr 7
p = 7
QR_p (nonzero squares) = {1, 2, 4}
Residue -> square roots modulo p
0 -> {0} (zero is listed separately)
1 -> {1, 6}
2 -> {3, 4}
4 -> {2, 5}

$ ./build/elliptic-curve curve 7 2 3 build/puntos-p7.txt
p = 7
a = 2
b = 3
E:  $y^2 = x^3 + a*x + b \pmod{p}$ 
Rational points (x, y, z):
(2, 1, 1)
(2, 6, 1)
(3, 1, 1)
(3, 6, 1)
(6, 0, 1)
(0, 1, 0)
Total points (including infinity): 6
Saved to: build/puntos-p7.txt

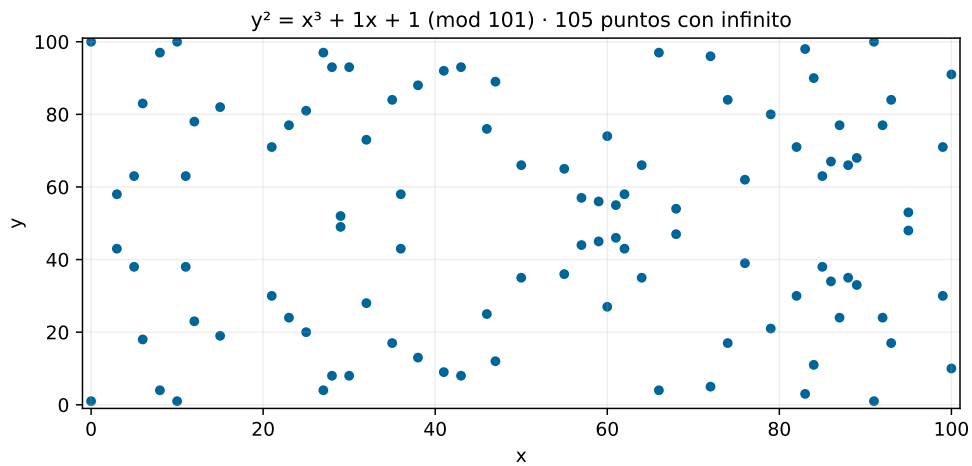
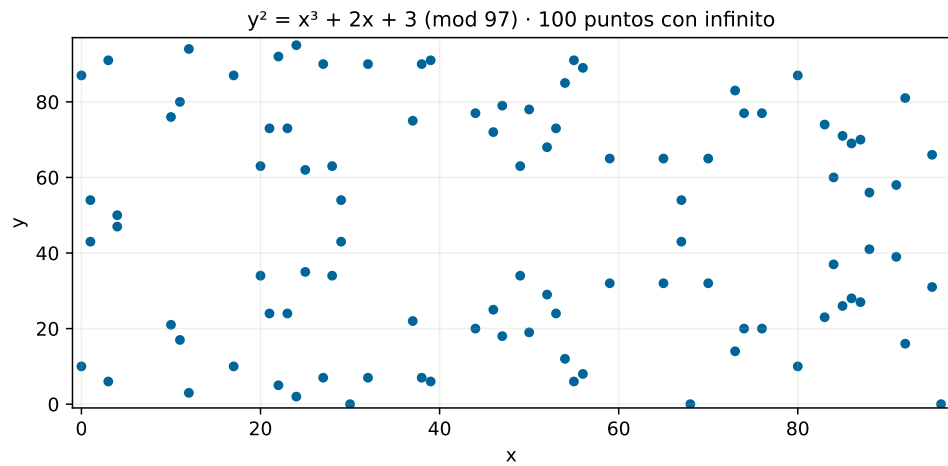
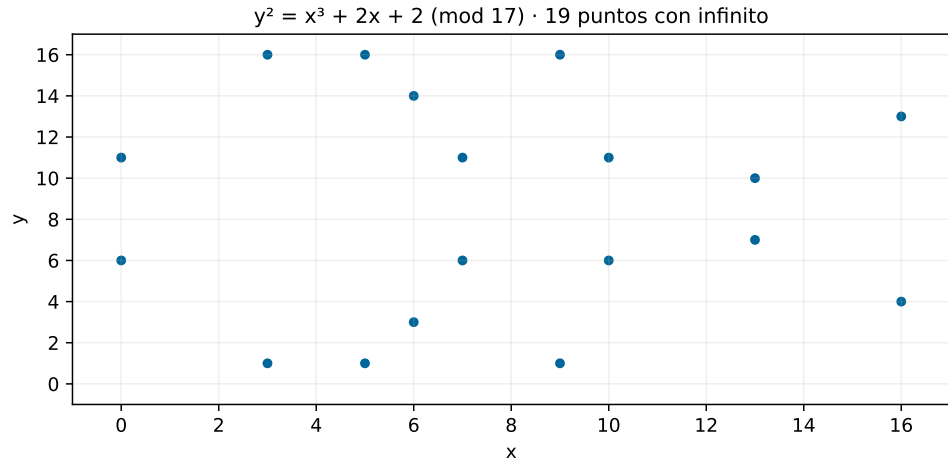
$ ./build/elliptic-curve generate 16 20260924
bits = 16
seed = 20260924
Miller-Rabin: 32 rondas (primo probable)
p = 55871
a = 19560
b = 12704
 $4*a^3 + 27*b^2 \not\equiv 0 \pmod{p}$ 

$ ./build/elliptic-curve add 65537 1 1 49606 64426 1 2565 62370 1
P + Q = (43190, 62760, 1)

$ ./build/elliptic-curve double 65537 1 1 49606 64426 1
2P = (27838, 52313, 1)
```

3.1. Gráficas de puntos racionales

Se graficaron los archivos puntos-17.txt, puntos-97.txt y puntos-101.txt. El infinito se cuenta, pero no se dibuja en el plano.



3.2. Curvas de 16, 32, 64 y 512 bits

En todos los casos: $y^2 = x^3 + ax + b \pmod{p}$ y $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

Captura de consola

```
16 bits · semilla 20260924
p = 55871
a = 19560
b = 12704

32 bits · semilla 20260940
p = 3322834597
a = 2929689510
b = 1746233044

64 bits · semilla 20260972
p = 16057177724199694949
a = 2729491702699733433
b = 6211698218018205146

512 bits · semilla 20261420
p =
11320044845349622089693698352800981109127033374520462129386255250681965130308343341
438930404999424645812224529025601157087182348076548581544947502755847177
a =
26704643583417387649107720148671761628660266764163045469591458007221405316787300937
25022627749456226459108689712457303799761580469649134569128476632654046
b =
56060450608779411354996265401574514713510766327025672281772865518220199101074002855
66904539674880588176976624885431937697110441811435465611434008683745638
```

3.2. Curvas de 1024 y 2048 bits

Captura de consola

```
1024 bits · semilla 20261932
p =
13622871222846721312339586559157179301988979492260289661668323113654832947308242689
02141162213115782623930888316574520058508037132807539953449862000153490494317466233
21808418392948940600946302922547649660750869348064231878335827546454816437275167962
642373926747194972668432930082975364776918101277727613620367
a =
10709037835335369358588770272365383855095167563746450837655925475256408167177173737
48772504030072421004326598522660559571271388450149831454415381898139217303749344266
05286560050751932617616603723715210458676507444060966766082496814346997480776980431
683360313667843193548133243120127868965245916810374106863005
b =
19747194615339395098256527512278899589361611949921691669922280629519220327943980207
47906918211587383772025411153479663220657290141031348660065033504022463327399652057
02590949269496749194827865346600714574484633140427397724482595667369304748568203108
61922649001513299337438838368180313812582068231640245699194
```

Captura de consola

```
2048 bits · semilla 20262956
p =
20393373469819914708276195225024032417151758183004474808475900028253550239008627794
48075425809829740007650884571825607871496220183623729694757694329870595572815737467
79362870619577452103956160238049108810447852152853065394557678301983650688285074613
47830565804741695914090080675981646810114654793804690626282188432928642835949201977
23741919999876700540155903706905421513830531751016422920565117062476343812304011152
25178571205038602344239113697895460187973756134183117741922481594553387331645078493
29767261270631615423845721092580126277609640835044298639522925865255548787767651936
213860970120835737622055523088908269
a =
54756797156583360032845563742721336055821854213313526160893375256502441258066295578
70555367547348121165396008285946494569410438446929758458367893347951824427867863752
59051970349022011020297588958953597148802947082611648571290501553492268739889080138
67927859088614977303918925508866015808817185164294506581060934037474896067206584143
47778730375449793747358191334002890118052533065465619699592031401094712040621534478
30200617686552878482342283771361988352787316766599502528745123552261522787660062161
23970613830664679887312475838778288333477609139798368078849185661630392526390082270
73633885874319360640867719429432703
b =
55466169398692667846558156426070194253926932035462769991437748729848841192111442115
10127471525728321184532216393386115326237121860667014919222900839082714407460739925
72445196149904827217854072745106412966719702481041963488822180128726934288767397457
8070244962743985851448842847468322028135155228604831405107633937911486306332922517
26262342881302967135243936656583590641145021491357210892051876427258343471012922766
97445961246086288052964164597111246253112988326385429295390335124593745386786853114
12428603344296909271049734498795158536720619764971147474866645640858247269866548806
77778286349133920952546665362974250
```

3.3. Resultados de las operaciones

Se usaron los puntos P y Q de los cuatro incisos del enunciado.

Captura de consola

```
Caso a) p = 65537, a = 1, b = 1
P+Q = (43190, 62760, 1)
2P = (27838, 52313, 1)
2Q = (11605, 53046, 1)

Caso b) p = 4294967311, a = 30, b = 97
P+Q = (1365729123, 1751038107, 1)
2P = (4068212755, 1475533388, 1)
2Q = (1572879236, 1387343506, 1)

Caso c) p = 2147483647, a = 125, b = 2
P+Q = (438842645, 2065604505, 1)
2P = (742142419, 1577310052, 1)
2Q = (1125939014, 778061759, 1)

Caso d) p = 2305843009213693951, a = 1, b = 1300
P+Q = (2223280671137397525, 2193609183055342937, 1)
2P = (492215062149154878, 1699466406640207765, 1)
2Q = (1425847782956718139, 2293499795315943298, 1)
```

Fórmulas consultadas: [Ben Lynn](#), suma y duplicación. Prueba de primalidad: [Handbook of Applied Cryptography](#), algoritmo 4.24.

Captura de la web: puntos

01 / $\mathbb{QR}_p$

RESIDUOS CUADRÁTICOS

Encuentra cada residuo y todas sus raíces módulo p.

Primo p

7

Calcular residuos

```

p = 7
QR_p (nonzero squares) = {1, 2, 4}
Residue → square roots modulo p
0 → {0} (zero is listed separately)
1 → {1, 6}
2 → {3, 4}
4 → {2, 5}
    
```

02 / $E(\mathbb{F}_p)$

PUNTOS RACIONALES

$y^2 \equiv x^3 + ax + b \pmod{p}$. El conteo incluye (0, 1, 0).

Primo p a b

7 2 3

Enumerar puntos

```

p = 7
a = 2
b = 3
E: y^2 = x^3 + a*x + b (mod p)
Rational points (x, y, z):
(2, 1, 1)
(2, 6, 1)
(3, 1, 1)
(3, 6, 1)
(6, 0, 1)
(0, 1, 0)
Total points (including infinity): 6
Saved to: elliptic-curve-points.txt
    
```

[Descargar puntos \(.txt\)](#)

Captura de la web: operaciones

ARITMÉTICA DE CURVAS ELÍPTICAS

Enteros de hasta 2048 bits. Usa decimales y puntos (x, y, 1); el infinito es (0, 1, 0). En suma y duplicación se supone p primo y se verifica que los puntos pertenezcan a la curva.

GENERAR UNA CURVA

Miller-Rabin: 32 rondas, primo probable. Aleatoriedad de práctica. La semilla permite repetir la corrida en el mismo entorno.

Bits de p

16

Semilla

20260908

Generar curva

```
bits = 16
seed = 20260908
Miller-Rabin: 32 rondas (primo probable)
p = 60443
a = 40776
b = 10438
 $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ 
```

P + Q

p

65537

a

1

b

1

P (x, y, z)

x

49606

y

64426

z

1

Q (x, y, z)

x

2565

y

62370

z

1

Sumar puntos

P + Q = (43190, 62760, 1)

2P

p

65537

a

1

b

1

P (x, y, z)

x

49606

y

64426

z

1

Duplicar punto

2P = (27838, 52313, 1)